The HexaPedal

Section HM

Phase III - 2025-11-06

Mohamad Addasi (40278616)

Fouad Elian (40273045)

Junior Boni (40287501)

Tristan Girardi (40272203)

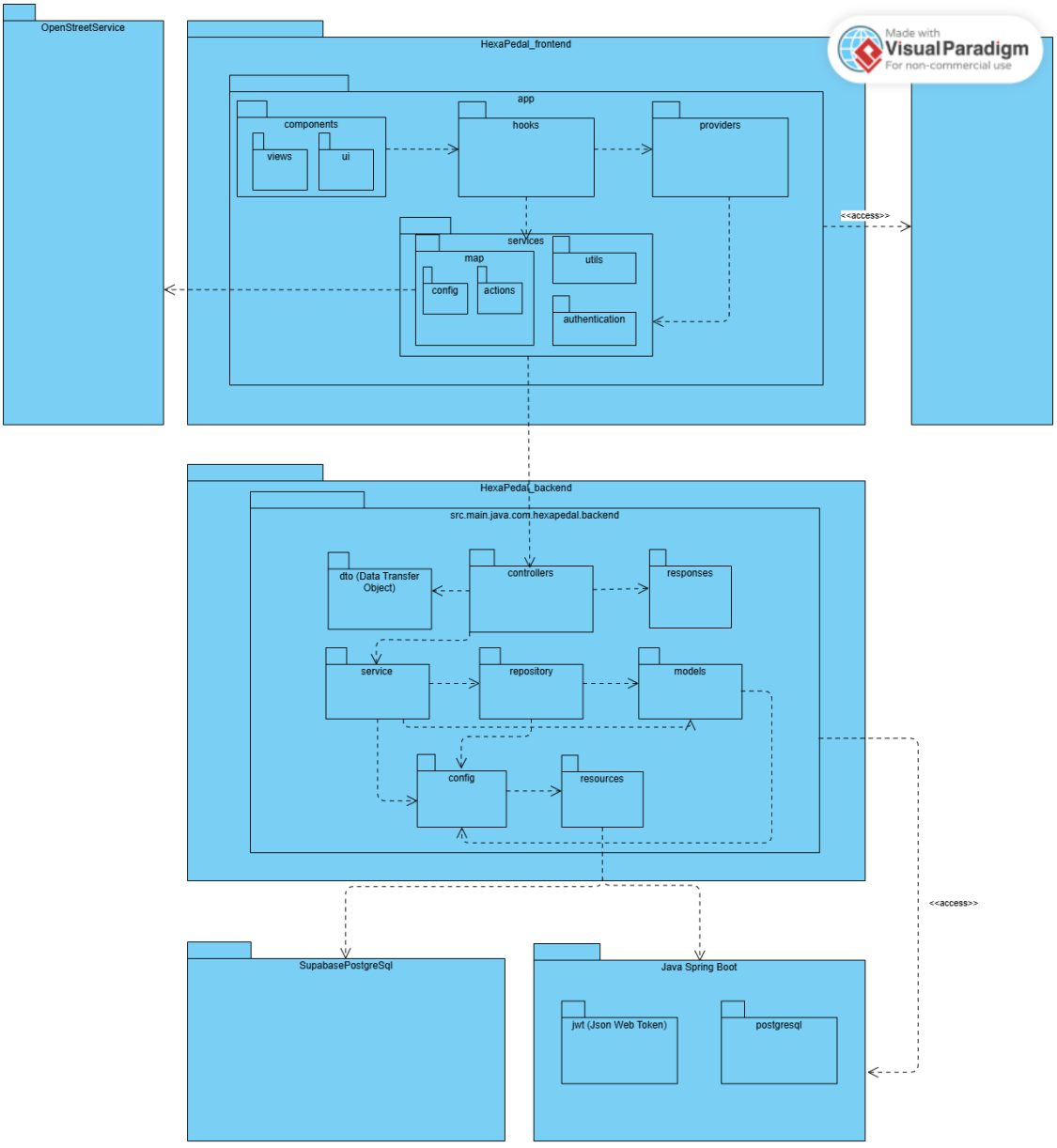
Youssef Yassa (40265083)

Youssef Youssef (40285384)

Table of Contents

System Architecture – *No Critical Changes*

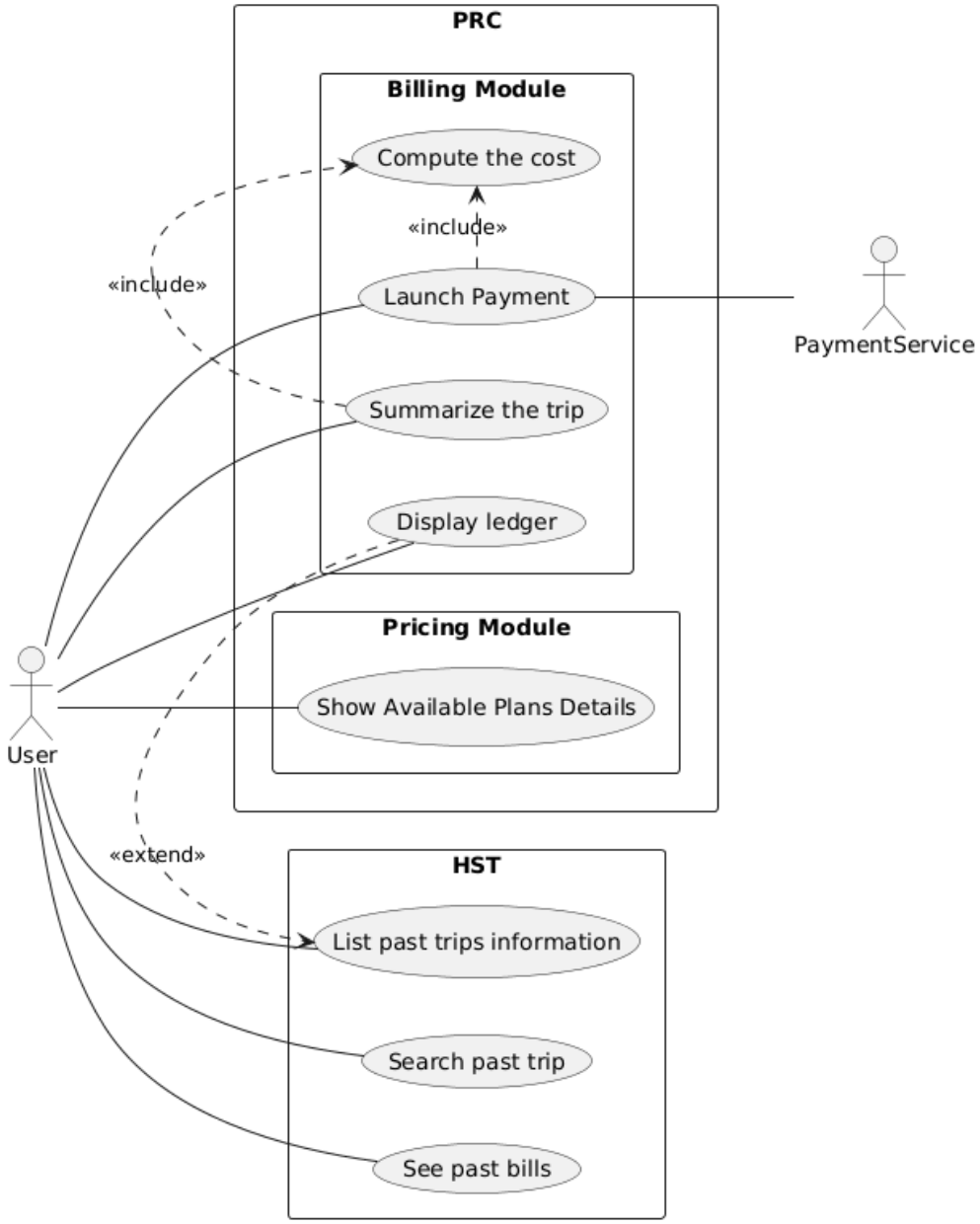
Overview



We have not introduced any critical changes from phase 2 to phase 3.

Use Case Model

Use Case Diagram



Written Use Cases for PRC module

Show Available Plan Details

- **ID**: R-PRC-1
- **Title**: Display plans'/rides pricing for all users
- **Description**: Users get to see the pricing for a selected ride (which would be affected by the type of bike chosen), and for subscription plans.
- **Primary Actor**: User
- **Preconditions**:
    - Pricing for rides has been set.
    - Pricing for subscriptions has been set.
- **Postcondition**:
    - User views the rides' price.
    - User views the subscriptions' price.
- **Inputs**:
    - N/A
- **Outputs**:
    - N/A
- **Main Success Scenario**:
    - User accesses the platform.
    - User views the rides' price and the subscriptions' price.

Compute the Cost

- **ID**: R-PRC-2
- **Title**: Compute Cost
- **Description**: Upon displaying the summary of a trip, the PRC system accounts for the trip information and the user's subscription (if any).
- **Primary Actor**: User
- **Preconditions**:
    - User completed at least one trip.
    - The trip has valid information.
- **Postconditions**:
    - The price of the trip is calculated by factoring the ride's type and the user's subscription (if any).
- **Inputs**:
    - Trip summary
    - User's subscription
- **Outputs**:

    - The trip’s price.
- **Main Success Scenario**:
    - User records a trip.
    - User selects a valid ride type.
    - System computes and displays the trip’s price

Summarize the Trip

- **ID**: R-PRC-3
- **Title**: Summarize the Trip
- **Description**: After returning the bike, the system displays the trip’s summary containing some information such as the bike type.
- **Primary Actor**: User
- **Preconditions**:
    - User completed a trip.
    - User used a valid ride.
- **Postconditions**:
    - Display a summary of the trip.
- **Inputs**:
    - Ride’s information
    - Trip’s information
- **Outputs**:
    - List of trip's information
- **Main Success Scenario**:
    - User returns a bike.
    - System displays a summary of the trip’s information

Display Ledger

- **ID**: R-PRC-4
- **Title**: Display Ledger
- **Description**: On the user dashboard, list a subset of the user billing and trips history.
- **Primary Actor**: User
- **Preconditions**:
    - User has a valid session (not necessarily authenticated).
    - User completed at least one trip and/or payment.
- **Postconditions**:
    - Dashboard displays a subset of the billing and trips history.
- **Inputs**:

    - User's trips and billing history
- **Outputs**:
    - Subset of the user's trips and billing history
- **Main Success Scenario**:
    - User accesses the application.
    - User's dashboard loads a lightweight version of the complete history.

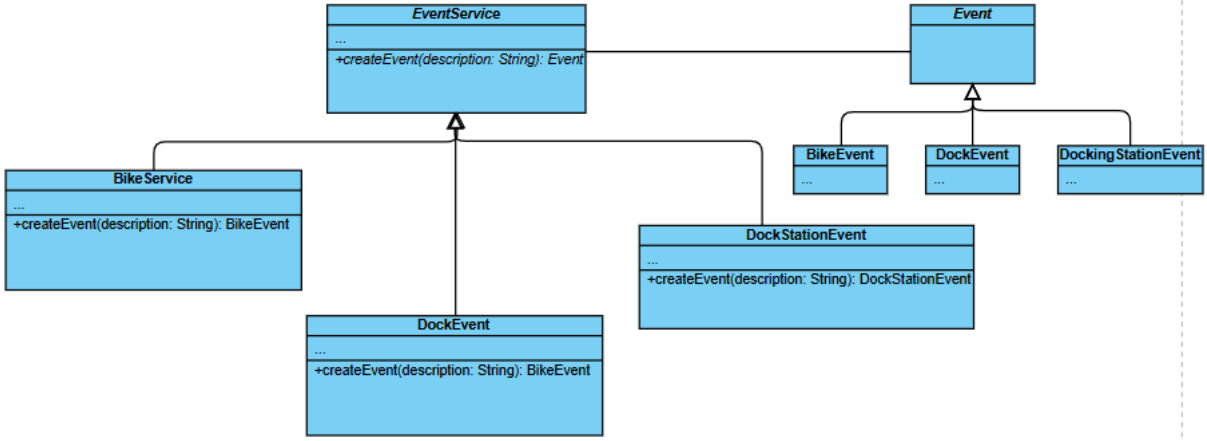
Launch Payment

- **ID**: R-PRC-5
- **Title**: Launch Payment
- **Description**: Upon payment request, the PRC system handles the payment logic by communicating with the required service with regard to the payment method.
- **Primary Actor**: Rider
- **Preconditions**:
    - User requested to make a payment.
    - User handled valid payment information.
- **Postconditions**:
    - Subsystem redirects the request to the relevant external payment service.
- **Inputs**:
    - User's payment information
    - Computed Price
- **Outputs**:
    - The payment service confirming or infirming the success of the request.
- **Main Success Scenario**:
    - User enters his payment information.
    - User requests to pay.
    - PRC parses the payment methods.
    - PRC forwards the requests to the relevant payment service.
    - Payment service handles the payment request.
    - Payment request sends a validation message confirming payment success.

Design pattern implementations

Events - Factory Method

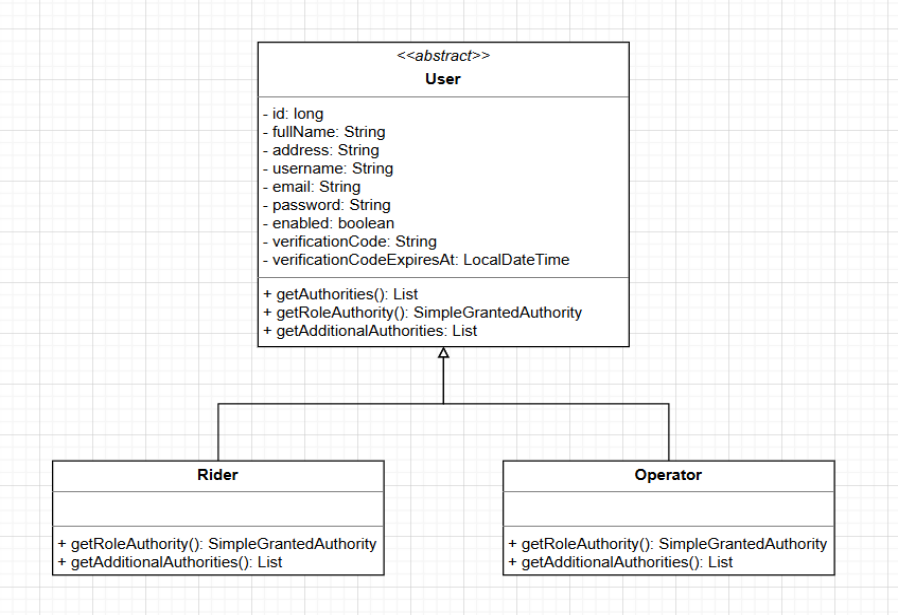
For the creation of our events, we have opted to use the factory method design pattern because it simplifies the creation of the events and makes it much more maintainable as we are ensuring the single responsibility principle.



Template

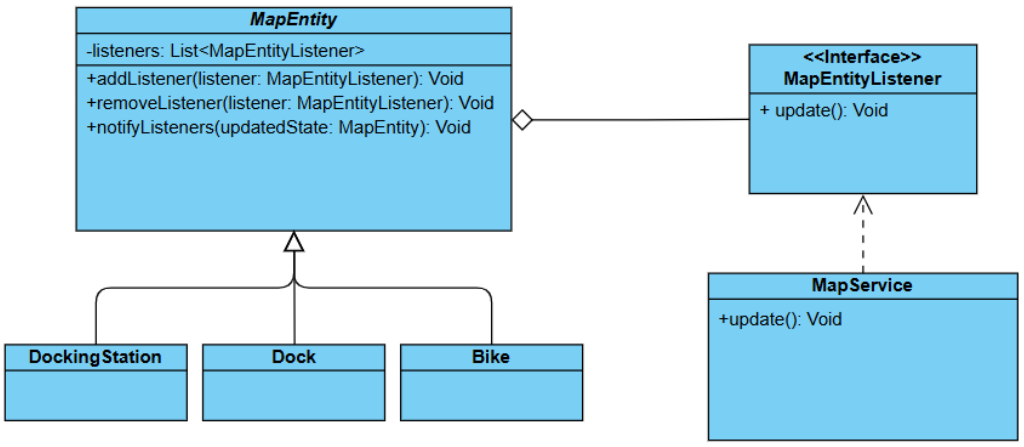
In our system, we have two concrete user roles: Rider and Operator. Each user's role has its own permissions on what they can or cannot do. This needs to be verified upon logging in to the application. Both roles have authorities, but none of these concrete implementations should ever override how the assignment or retrieval of the authorities gets done. Some methods that would be okay to override would be to simply fetch role specific authorities such as the Rider being able to reserve a bike and Operator being able to change

the dock status. But the main method to set or fetch the general authorities should be done by the parent class, that is our template method.



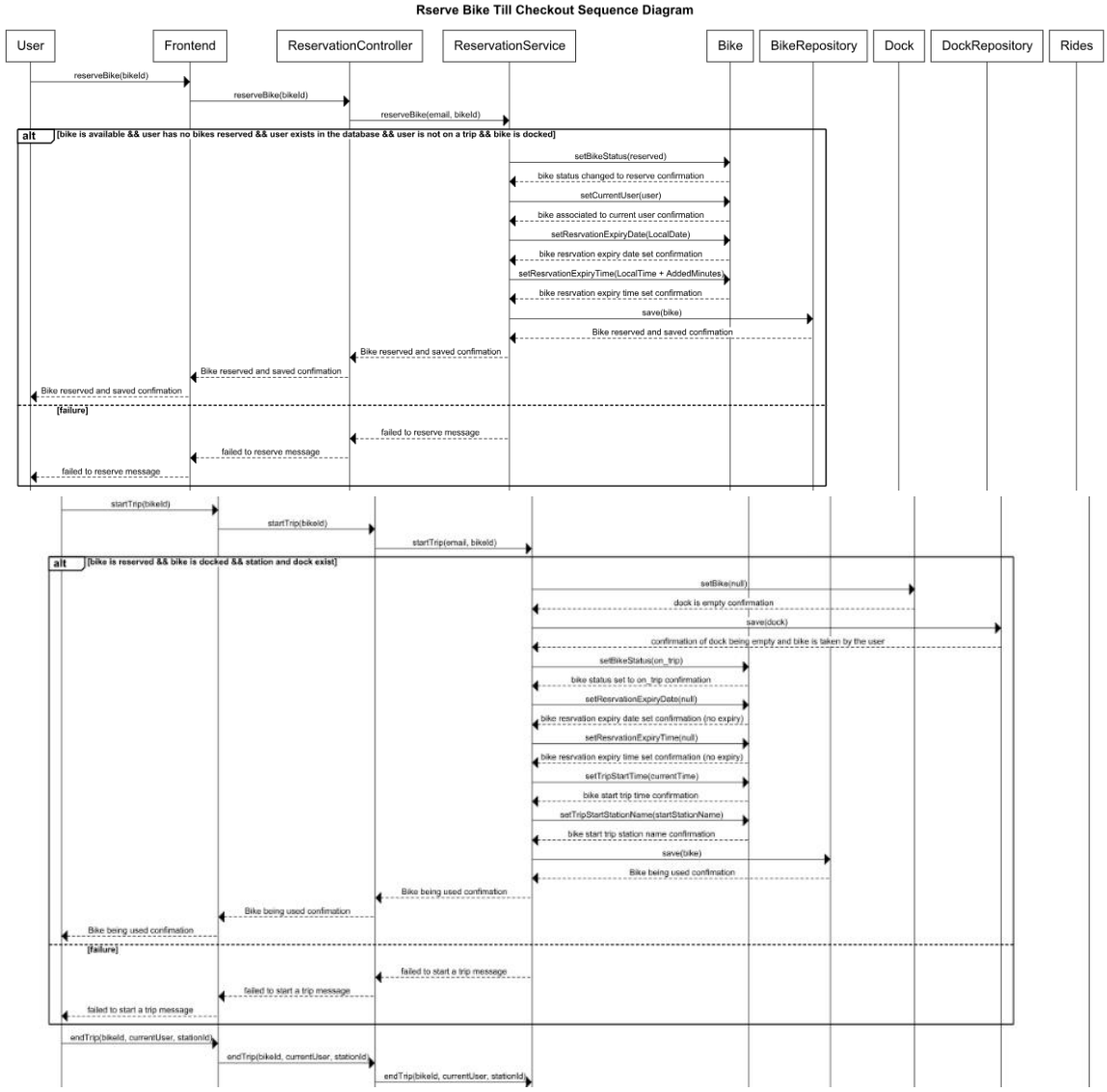
Map - Observer pattern (phase 2 revision)

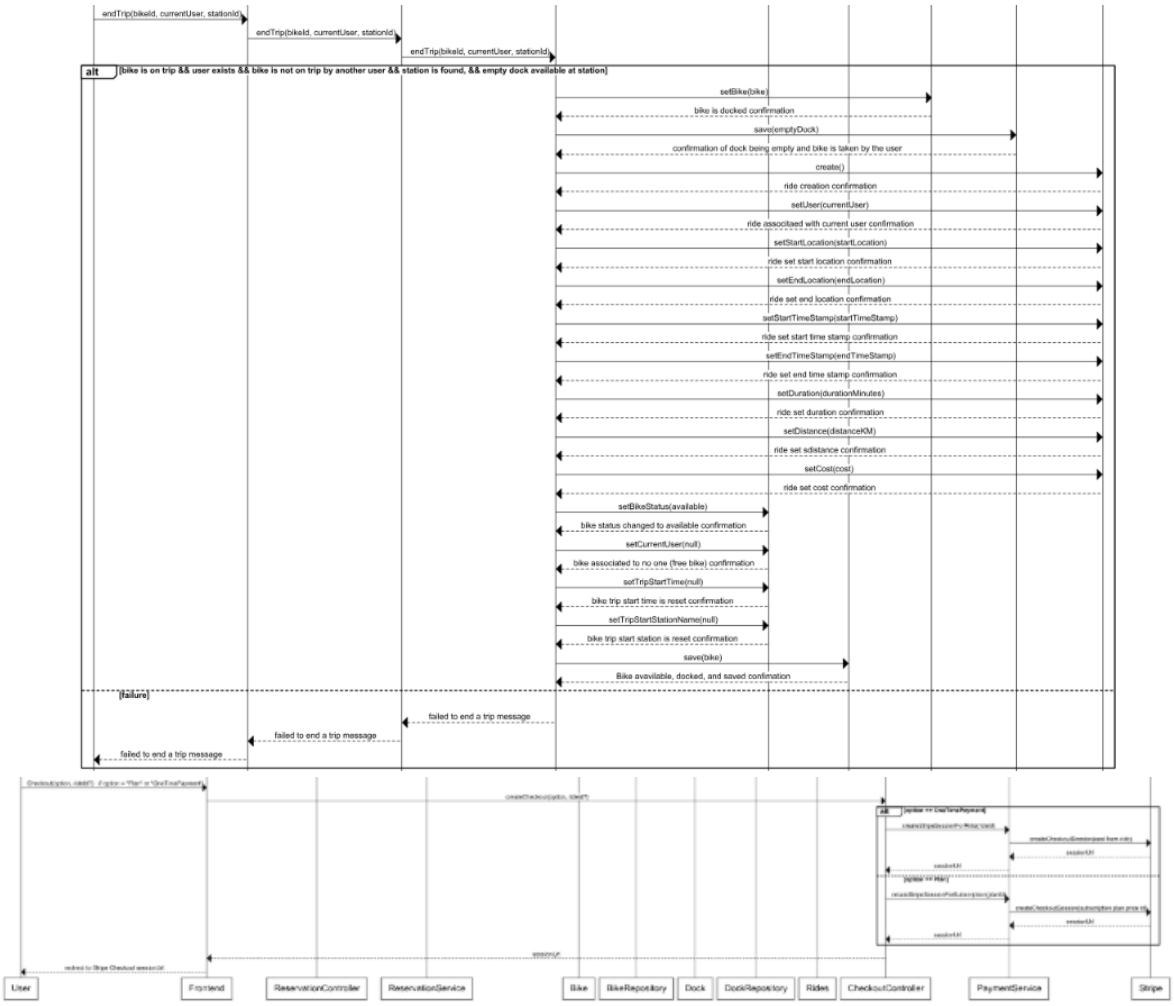
In the previous version of our documentation, we introduced the observer pattern with the publisher being the map and the clients being the subscribers. Although at the time, it made sense in theory, it only represented Java SpringBoot's WebSocket functionality. In other words, it did not affect our model and business logic. To resolve such issue, we refined the pattern so that the service class is the observer as it is the one responsible to trigger a specific update property: it is the one responsible to utilize SpringBoot's WebSocket to broadcast changes to a given endpoint. Additionally, we realized in our revision that the the map is not really the publisher. Rather, it is the different map entities (i.e, dock stations, docks, bikes, etc) that publish their changes and upon changes, they should update (by broadcasting) the changes to users.



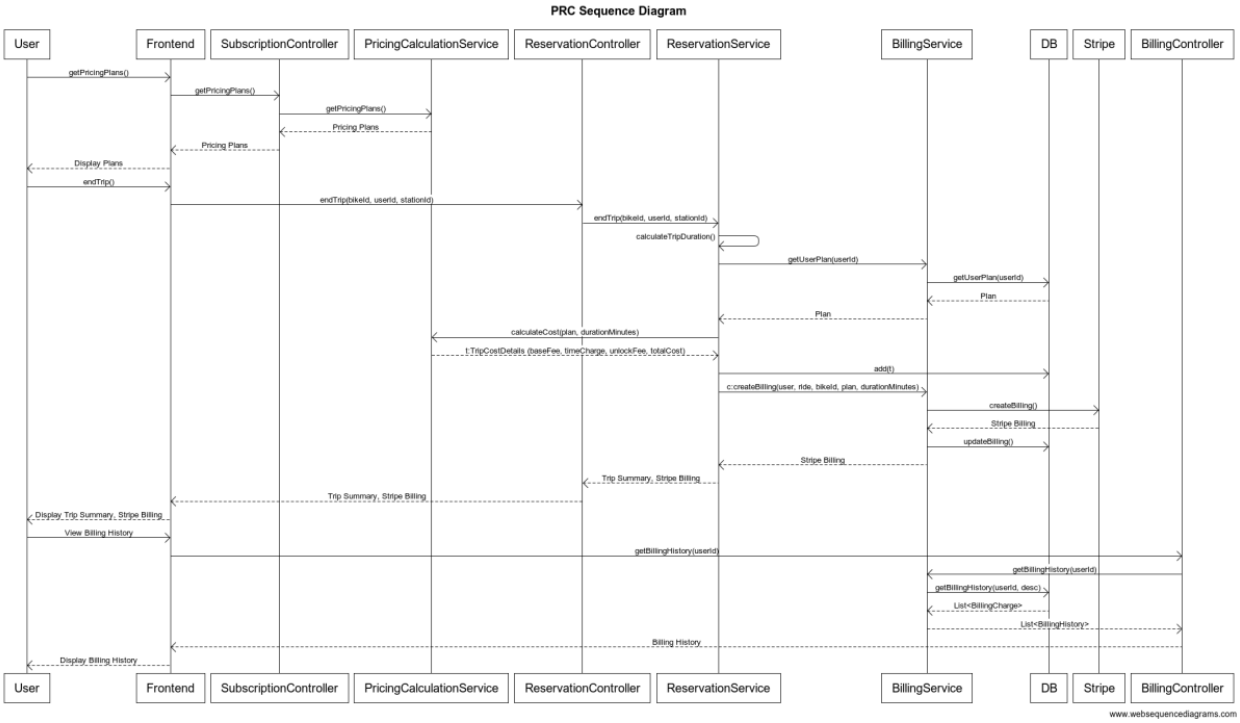
Sequence Diagrams

Sequence diagram for the rider's interaction with the Dashboard UI when reserving a ride, until checkout



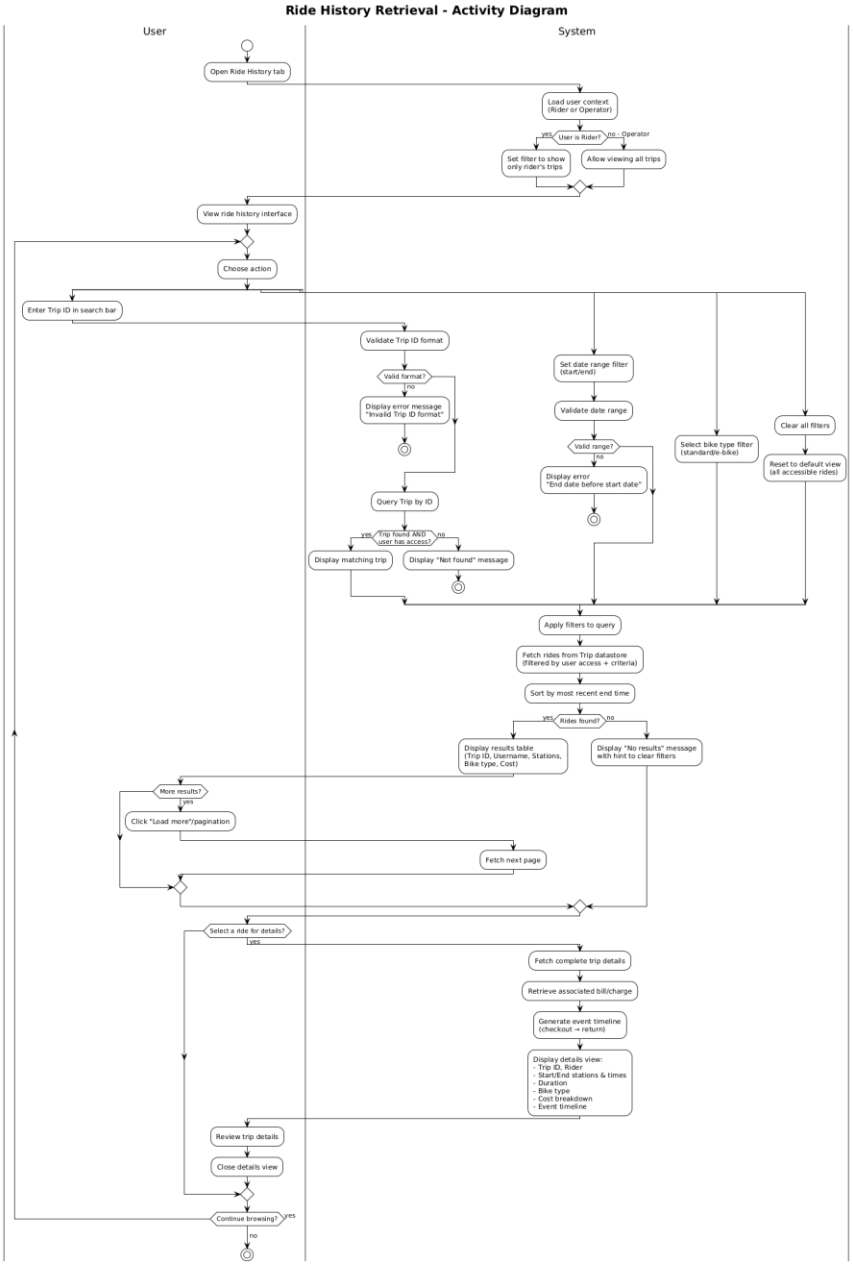

endTrip(bikeId, currentUser, stationId)
endTrip(bikeId, currentUser, stationId)
endTrip(bikeId, currentUser, stationId)
alt
[bike is on trip && user exists && bike is not on trip by another user && station is found, && empty dock available at station]
setBike(bike)
bike is docked confirmation
save(emptyDock)
confirmation of dock being empty and bike is taken by the user
create()
ride creation confirmation
setUser(currentUser)
ride associtaed with current user confirmation
setStartLocation(startLocation)
ride set start location confirmation
setEndLocation(endLocation)
ride set end location confirmation
setStartTimeStamp(startTimeStamp)
ride set start time stamp confirmation
setEndTimeStamp(endTimeStamp)
ride set end time stamp confirmation
setDuration(durationMinutes)
ride set duration confirmation
setDistance(distanceKM)
ride set sdistance confirmation
setCost(cost)
ride set cost confirmation
setBikeStatus(available)
bike status changed to available confirmation
setCurrentUser(null)
bike associated to no one (free bike) confirmation
setTripStartTime(null)
bike trip start time is reset confirmation
setTripStartStationName(null)
bike trip start station is reset confirmation
save(bike)
Bike avavilable, docked, and saved confimation
[failure]
failed to end a trip message
failed to end a trip message
failed to end a trip message
User
Frontend
ReservationController
ReservationService
Bike
BikeRepository
Dock
DockRepository
Rides
CheckoutController
PaymentService
Stripe

Sequence diagram for PRC

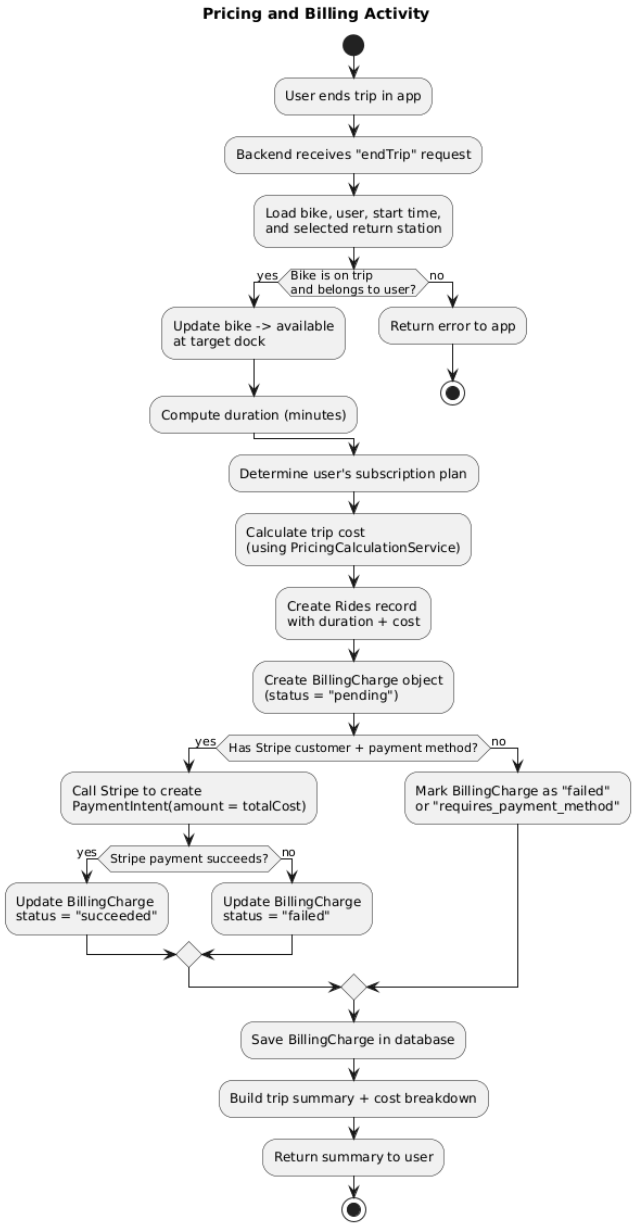


Activity Diagrams

Activity Diagram - Retrieve Ride History



Activity Diagram - PRC



Contribution Self-Declaration

All contributing members have contributed equally to this Phase delivery